

Human-centered Machine Learning Project Report 2025 @ UU

Julia Baas
j.n.baas@students.uu.nl
Utrecht University
Utrecht, the Netherlands

Ali Nesaei
a.nesaei@students.uu.nl
Utrecht University
Utrecht, the Netherlands

Bar Melinarskiy
b.melinarskiy@students.uu.nl
Utrecht University
Utrecht, the Netherlands

Youssef Ben Mansour
y.benmansour@students.uu.nl
Utrecht University
Utrecht, the Netherlands

ABSTRACT

This project examines the potential of interpretable methods in an NLP task and how they impact model accuracy. We approached this by predicting drug review ratings using four models: Lasso Regression, LightGBM, BERT, and ProtoLM. Each model was assessed for both predictive performance and interpretability using the Quantus evaluation framework. While BERT achieved the highest accuracy, it lacked interpretability. Contrary to expectations, Lasso fell short on interpretability. LightGBM performed moderately on both interpretability and accuracy, while ProtoLM performed poorly overall. Our findings highlight the difficulty of building models that are both accurate and interpretable in NLP tasks and underscore the need for better inherently interpretable approaches.

ACM Reference Format:

Julia Baas, Bar Melinarskiy, Ali Nesaei, and Youssef Ben Mansour. 2025. Human-centered Machine Learning Project Report 2025 @ UU. In *Proceedings of Utrecht University (INFOMHCLM'2025)*. Utrecht University, 9 pages.

1 INTRODUCTION

Online medication reviews provide valuable insights into patient experiences, yet extracting meaningful patterns from these texts to predict user ratings remains challenging. This work examines how model choice and feature representation affect both predictive performance and interpretability in medication rating prediction.

Following Cynthia Rudin [1], who advocates for interpretable models over black-box approaches, we compare methods across the interpretability spectrum: transparent models like Linear Regression, moderately interpretable Gradient Boosting Machine (GBM), black-box fine-tuned BERT, and Proto-lm - a deep model with built-in explanation mechanisms [2].

We employ distinct feature extraction methods tailored to different model architectures: TF-IDF vectorization for traditional machine learning models and BERT embeddings for deep learning approaches. This dual methodology allows us to leverage both interpretable sparse features and dense contextual representations. To address the inherent opacity of these models and feature extraction methodologically, we apply SHAP (SHapley Additive exPlanations)

[3] and LIME (Local Interpretable Model-agnostic Explanations) [4] for post-hoc interpretability analysis. We systematically evaluate explanation quality across five key metrics: complexity, sparsity, robustness, maximum sensitivity, and faithfulness, using Quantus [5] an established evaluation framework. Our complete implementation, including model architectures, explanation methods, and evaluation metrics, is publicly available at [GitHub](#).

Following the implementation, several interesting findings emerged. Notably, the BERT model achieved the best overall performance, while the Proto-lm model performed the worst. In terms of interpretability, the analysis revealed some unexpected results, particularly the difficulty in understanding the features used by the linear model. This complexity stems from the use of SVD, which transforms the original features into abstract components, as well as the fact that these components were manually labeled, complicating their interpretability.

2 RELATED WORK

This project is inspired by a paper by Cynthia Rudin [1]. In her work, Rudin makes an interesting case: instead of building complex "black-box" machine learning models and then trying to explain them afterward, we should aim to create models that are inherently interpretable from the start. She argues that uninterpretable systems simply should not be used in critical, "high-stakes" areas like criminal justice, healthcare, or finance. To support her claim, she points out the problems with black-box models that require post hoc explanations. For example, she shows that those post hoc explanations are inherently unreliable: if they'd match the model's internal workings, they'd be the model. Inherently interpretable models, on the other hand, offer faithful explanations, are transparent, and are often just as accurate as black-box models. However, Rudin points out the challenges that are faced when trying to implement an inherently interpretable model. For example, it can take significant effort to construct an interpretable model without the use of black-box models that are complex but easy to implement. Rudin concludes that the research community should change course: shifting focus from using opaque black-box models to developing better inherently interpretable models.

Although Rudin's work focuses on areas outside of Natural Language Processing (NLP), her arguments inspired us to explore similar questions in NLP. Is it possible to find inherently interpretable models that are just as accurate as black-box models for an NLP

This paper is published under the Creative Commons Attribution-NonCommercial-NoDerivs 4.0 International (CC-BY-NC-ND 4.0) license. Authors reserve their rights to disseminate the work on their personal and corporate Web sites with the appropriate attribution.

INFOMHCLM'2025, 2025, Utrecht, the Netherlands

© 2025 Utrecht University

ACM ISBN xxxxxxxx.

<https://github.com/BarGinger/HCLM-NLP-Project>

task? Since Rudin does not answer this, our project aims to find out if her conclusions also apply to NLP.

The question of model interpretability in NLP has become increasingly important as these systems are deployed in high-stakes applications like resume screening, automated translation, and conversational agents. Despite growing research interest, the field still lacks widely accepted standards for evaluating interpretability methods.

Jain and Wallace (2019) [6] raised fundamental questions about one of the most commonly cited explanations in modern NLP models. They showed that attention mechanisms in transformer-based models, often presented as natural explanations for model behavior, are not reliable indicators of how these models actually make decisions. By creating counterfactual attention patterns that left predictions unchanged while dramatically altering attention weights, they demonstrated that attention can be manipulated without affecting model outputs. This finding supports Rudin's broader argument that many post hoc interpretability methods create an illusion of understanding rather than providing genuine insights into model decision processes.

Building on this foundation, Madsen et al. (2022) [7] conducted a comprehensive survey of post-hoc interpretability techniques for neural NLP models, systematically organizing the various approaches researchers have developed for understanding deep learning architectures. While their survey focused primarily on neural methods, our work extends this perspective to include traditional machine learning approaches such as Linear Regression and LightGBM. This broader scope acknowledges that interpretability challenges exist across the entire spectrum of NLP modeling techniques, not just in deep learning systems.

3 DATASET AND METHODOLOGY

Our project utilizes a Kaggle drug review dataset [8] containing user reviews of drug treatments. It comprises six key features: patient identification, drug name, associated medical condition (e.g., birth control, anxiety), free text review, numerical rating (1-10 scale where 10 is most helpful), review date, and usefulness votes. A glimpse of it can be seen in Fig. 1. Our goal is to predict review scores using just the review text, excluding all other available features. The dataset is already split into train (196,397 samples), validation (27,704 samples), and test (65,784 samples).

Detail	Compact	Column
review_id	10	10
drugname	10	10
condition	10	10
review	10	10
rating	10	10
usefulCount	10	10
review_length	10	10

review_id	drugname	condition	review	rating	usefulCount	review_length
163748	Mirtazapine	depression	"I've tried a few antidepressants over the years (citalopram, fluoxetine, escitalopram), but none work."	10.0	22	68
286473	Prozac	chronic's disease, maintenance	"My son has chronic's disease and has done very well on the Prozac. He has no complications and when he..."	9.0	17	48
39393	Exemore	weight loss	"Exemore combines drugs that were used for diabetes, smoking, and sexual dysfunction. Please use this..."	9.0	35	143
97768	Optiflow 1 / 35	birth control	"I have been on this birth control for one month after reading some of the reviews on this type of..."	9.0	4	149

Figure 1: Drug review Dataset

3.1 Preprocessing

To make the raw data ready for feeding the models, we created a pre-processing pipeline. The target of this step was to remove noise and ineffective data, then standardize the text and make it clear and structured so the models could learn meaningful patterns better. The process contains these steps:

- (1) Contraction Expansion: In the first step, we applied expansion. Terms like "I'm" became "I am". This helps with word assimilation.
- (2) Lowercasing: All text was lowercased. This step helps count words like "Good" and "good" as similar words.
- (3) Removing URLs and Emails: In this step, every URL and email in the comments was removed.
- (4) Removing Special Characters: All special characters that were not letters or numbers (e.g., #, , etc.) were removed in this step.
- (5) Advanced Pre-processing using spaCy: For the main steps of the NLP task, we used the spaCy package. To make it faster, batch processing using `nlp.pipe()` was considered. This step contains these parts:
 - Lemmatization: Every word was converted to its root. For example, words like "working" and "works" were both converted to "work". This helps reduce the number of unique words.
 - Stopword Removal: Removing common words that don't add any information to the sentence, for example: the, a, and is.

After applying these steps, a new column containing the clean text was added to our dataframe to be the main input for the next steps, which are feature extraction and training the models.

3.2 Feature Extraction

The feature extraction pipeline, as described in Fig. 2, converts preprocessed text into appropriate inputs for our chosen models. For the traditional ML models (Linear Regression and LightGBM), which require numerical rather than textual input, TF-IDF was used in order to create the word embedding. This method is a commonly used technique that measures how important a word is to a document. It assigns higher scores to words that appear frequently within a specific document but are rare across the entire collection. This prioritizes unique and meaningful terms over common, less informative words. Then, in order to create a more compact feature set, SVD dimensionality reduction was applied to these combined features. This reduced the dimensionality to 200 SVD components. This technique was applied because having a large number of features significantly reduces the interpretability of the model. Additionally, sentiment features were also extracted from the text using sentiment analysis, which generates numerical scores for positive, negative, and neutral sentiment along with a compound sentiment score.

The SVD components, sentiment features, and length of the review were fed into the Linear Regression and LightGBM algorithms, which identified the fifteen most important features (consisting of thirteen SVD components, the sentiment feature, and the review length). The SVD components were manually labeled while using

an LLM model for inspiration. After that, the model was trained again on these fifteen most important features. The transformer-based models (BERT and Proto-lm) took a different approach by directly using the preprocessed text alongside the sentiment analysis features.

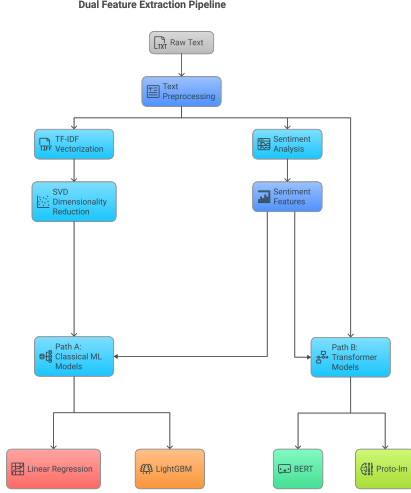


Figure 2: Feature Extraction Pipeline. Feature extraction pipeline for predicting drug review ratings from review text. Raw text undergoes preprocessing, then splits into TF-IDF vectorization and sentiment analysis. TF-IDF features are reduced using SVD. Two model paths emerge: classical ML models (Linear Regression, LightGBM) use SVD features combined with sentiment features, while transformer models (BERT, Proto-lm) use preprocessed text combined with sentiment features to predict review scores.

3.3 Quantus Metrics

In order to better understand the analysis of our evaluation results, we provide here a brief overview of each metric used from the Quantus framework.

Complexity – quantifies the difficulty of understanding an explanation by measuring the concentration of attribution scores across input features. It is calculated as:

$$\text{Complexity} = \sum_{i=1}^n \frac{|a_i|}{\sum_{j=1}^n |a_j|} \log \left(\frac{|a_i|}{\sum_{j=1}^n |a_j|} \right)$$

where a_i represents the attribution score for feature i , and n is the total number of features. This metric is based on Shannon entropy, with lower values indicating better explanations as they suggest more concentrated, interpretable attributions. The range is $[0, \log(n)]$, where 0 represents perfect concentration on a single feature and $\log(n)$ indicates uniform distribution across all features.

Sparsity – measures how concentrated attribution scores are across features using the Gini coefficient. It evaluates whether explanations focus on a few important features rather than spreading attribution uniformly across all inputs. Higher values indicate better sparseness, with the range $([0, 1])$ where values near 1 suggest concentrated explanations and values near 0 indicate uniform distribution across features. It is calculated as:

$$\text{Sparseness} = \frac{\sum_{i=1}^n (2i - n - 1) \cdot a_i^{\text{sorted}}}{n \cdot \sum_{j=1}^n a_j^{\text{sorted}}}$$

where a_i^{sorted} represents the i -th element of the sorted attribution vector (in ascending order), and n is the total number of features.

Max-Sensitivity – evaluates the stability of explanations under small input perturbations by measuring the maximum change in attribution scores:

$$\text{Max-Sensitivity} = \max_{|\delta| \leq \epsilon} \frac{|a(x) - a(x+\delta)|}{|\delta|}$$

where $a(x)$ represents the attribution function, δ is a small perturbation, and ϵ defines the perturbation bound. Lower values are better, indicating more stable explanations that are robust to minor input changes. Values typically range from 0 to several orders of magnitude, with 0 representing perfect stability.

Robustness – evaluates explanation consistency across similar inputs using the Local Lipschitz Estimate. It measures the maximum ratio of explanation change to input change by perturbing the input multiple times and computing:

$$\text{Robustness} = \max_{i=1, \dots, n} \frac{|a(x) - a(x'_i)|_2}{|x - x'_i|_2}$$

where $a(x)$ represents the attribution for original input x , $a(x'_i)$ is the attribution for the i -th perturbed input x'_i , and n is the number of perturbation samples. The implementation applies Gaussian noise perturbations with standard deviation 0.1 and computes the Euclidean distance for both numerator and denominator. Lower values indicate better robustness, suggesting that explanations remain consistent when inputs are slightly modified. Values range from 0 (perfect stability) to potentially large positive numbers, where higher values indicate explanations that are highly sensitive to small input changes.

Faithfulness Correlation – The Faithfulness Correlation (sometimes also called fidelity) metric evaluates how well attribution scores reflect the actual impact of features on model predictions. It measures the correlation between feature importance scores and the magnitude of prediction changes when those features are perturbed:

$$\text{Faithfulness} = \text{corr} \left(\{|a_i|\}_{i \in S}, \{|f(x) - f(x^{(i)})|\}_{i \in S} \right)$$

where S is a subset of features, $f(x)$ is the model prediction, and $x^{(i)}$ represents the input with feature i replaced by a baseline value. Higher values are better, with the range $[-1, 1]$, where 1 indicates perfect positive correlation between attribution magnitude and prediction impact, and -1 indicates perfect negative correlation.

To facilitate meaningful comparison across all five metrics, we applied a normalization procedure that transforms all metrics (except MAE) to a 0-1 scale where higher values consistently indicate better explanation quality. For metrics where lower values originally indicated better performance (Complexity, Max-Sensitivity, and Robustness), we applied an inversion transformation:

$$\text{normalized_score} = 1 - \frac{\text{original_score} - \text{min_score}}{\text{max_score} - \text{min_score}}$$

For metrics where higher values already indicated better performance (Sparsity and Faithfulness Correlation), we used standard min-max normalization:

$$\text{normalized_score} = \frac{\text{original_score} - \text{min_score}}{\text{max_score} - \text{min_score}}$$

This normalization enables direct visual comparison across metrics in our results visualization, where consistently higher bars represent superior explanation quality across all dimensions.

4 MODELS

For this project, we chose to compare four different models, ranging from simple and interpretable to more complex and black-box. Specifically, we selected Linear Regression, a LightGBM model, a BERT model, and a Proto-lm model to evaluate their performance and interoperability. Here, we briefly review how each model works and describe the training process for each.

4.1 Linear Regression

Linear models belong to the class of machine learning models, widely used across several domains. The goal is to find a linear relationship between a set of independent variables (features) and a dependent variable which represents the *target* outcome to be predicted. The model's objective is to learn a specific coefficient (weight) for each feature, representing its individual contribution to the final prediction. This learning process typically involves an optimization step, most commonly the minimization of a loss function, which adjusts the model's weights to reduce the difference between predicted and actual observed values.

The interpretability of linear models is coming from two key attributes. Firstly, the features used in these models are often applied directly or go through transparent transformations, keeping their original semantic meaning. This allows for a clear understanding of what each input variable represents. Secondly, the direct accessibility of the learned weights (β values). These coefficients explicitly reveal the importance and direction of each feature's influence on the predicted outcome. This transparency allows to report the contribution of each input feature to a given prediction, therefore providing a clear and justifiable interpretation for the model's output.

In this study, a Lasso regression model was used for the predictive task. This linear model was selected for its efficacy in mapping a compact set of features to the target variable, while also benefiting from its ability to perform regularization through L_1 penalty. The input to the Lasso model consisted of a combined feature matrix resulting from the pre-processing method. The model's objective is to learn a set of optimal coefficients for these features in order to predict the target medication scores. To improve generalization and prevent overfitting, the regularization strength (α parameter) of the Lasso model was optimized. This hyperparameter tuning was performed using Optuna, an automatic hyperparameter optimization framework [9]. The optimization process aimed to minimize the Mean Absolute Error (MAE) on the validation set, ensuring the selection of the most robust model configuration before final evaluation on the test dataset.

4.2 LightGBM

LightGBM is an optimized implementation of the Gradient Boosting Decision Tree (GBDT) algorithm, which is a powerful and widely used ensemble method in machine learning. GBDT builds decision trees sequentially, where each new tree is trained to predict the residual errors of the previous one. While effective, this process can become computationally intensive when dealing with large datasets. LightGBM addresses this by significantly reducing the number of data instances and features considered during training,

resulting in a faster and more efficient version of GBDT without sacrificing accuracy [10].

One of LightGBM's advantages is its ability to rank features based on their usefulness in tree splits, allowing for the identification of the most informative features—such as specific SVD components—used in prediction. This enhances interpretability compared to black-box models like deep neural networks, as it offers insight into which features contribute most to the model's decisions.

In this study, LightGBM was chosen for its balance between predictive power and interpretability, sitting between linear models and opaque black-box models. Two LightGBM models were trained: the first was used to identify the top fifteen most important features, which included review length, general sentiment, and thirteen SVD components. The second model was trained using only these selected features to reduce dimensionality and improve interpretability. Both models used the same hyperparameter tuning approach as the Lasso model, utilizing Optuna for optimization [9]. To explain individual instances, Tree SHAP will be used, because it is the most suitable method for LightGBM, since it is designed for tree-based algorithms [11].

4.3 Bert

For our black-box models we used Bert. For implementation, we used the bert-base-uncased model from Hugging Face transformers library which already has a general understanding of English language. We configured the BertForSequenceClassification architecture for our regression task by setting num_labels=1, which tells the model to output a single rating score. This model was then fine-tuned on our dataset for three epochs using the AdamW optimizer and MSE loss.

The training process used a batch size of 16 and a learning rate of $2e-5$, which are standard values for BERT fine-tuning. We set the maximum sequence length to 128 tokens to balance between capturing enough context and computational efficiency. The model was trained on a GPU to speed up the process, with validation performed after each epoch to monitor performance using RMSE as the evaluation metric.

Since BERT is a black box with millions of parameters, its decision making is not transparent. This is why using explainability tools like LIME is necessary to understand its predictions.

4.4 ProtoLM

In order to incorporate a deep learning model with a built-in, integrated surrogate explanation mechanism, we included Proto-lm [2], introduced by Xie, Vosoughi, and Hassanpour (2023). Proto-lm is a framework designed to enhance interpretability in large language models by combining a pretrained transformer encoder (such as BERT or RoBERTa) with a prototypical classification layer. Its core idea is to ground predictions in a set of learned, representative examples, which are referred to as prototypes, each associated with a specific class. By comparing input representations with these prototypes, the model can generate predictions that are accurate and explainable, offering insight into the underlying decision process.

The architecture consists of two main additions on top of the base LLM. First, an attention mechanism identifies the most informative tokens in the input, allowing the model to focus on the parts most

relevant for classification. The resulting attention-weighted representation is then passed to a prototypical network, which compares it to a small set of learned prototype vectors for each class. During training, the model optimizes not only for classification accuracy but also for proximity between inputs and their corresponding class prototypes, while pushing them away from other-class prototypes. At inference time, the model's prediction is guided by similarity to these prototypes, providing a clear, example-based explanation for its decision. The authors demonstrate the usefulness of this approach across sentiment analysis, natural language inference, and commonsense reasoning, showing that Proto-lm achieves competitive accuracy while offering interpretable outputs grounded in semantic similarity.

We trained an enhanced Proto-lm model for drug review classification using a multi-stage optimization approach aimed at preventing model collapse and addressing class imbalance. The model architecture was based on a BERT-base-uncased encoder, followed by a prototypical layer with 100 learnable prototypes in a 768-dimensional hidden space, designed for 10-class rating prediction (ratings 1–10). To handle the inherent class imbalance in drug review data, we applied inverse frequency class weights computed as $w_i = \frac{N}{K \times n_i}$, where N is the total number of samples, K is the number of classes, and n_i is the number of samples in class i .

The training objective combined a weighted cross-entropy loss with label smoothing ($\epsilon = 0.1$) and a simplified prototype regularization term. Specifically, the total loss was defined as $L_{\text{total}} = L_{\text{CE}} + 0.1\lambda_0 L_{\text{proto}}$, where L_{CE} incorporates both class weights and label smoothing, and $L_{\text{proto}} = 0.001\|P\|_2$ applies L2 regularization to the prototype matrix P to prevent collapse.

To ensure training stability, we used conservative hyperparameters: a learning rate of 5×10^{-5} , cohesion-separation ratio of 0.3, prototype pressure $\lambda_0 = 0.05$, and batch size of 64. The model was trained for up to 10 epochs using PyTorch Lightning [12] with early stopping (patience = 3), triggered by stagnation in validation prediction diversity. Throughout training, we monitored classification accuracy, prediction diversity (defined as the ratio of unique predicted classes to total classes), and prototype utilization to ensure the model preserved meaningful multi-class behavior rather than collapsing into binary or low-cardinality predictions. Unfortunately, due to resource limitations, we were unable to fine-tune the model with Optuna as we did for the other machine learning models.

5 RESULTS

To comprehensively evaluate our four selected models, we evaluated both their predictive performance and the quality of their explanations. For performance evaluation, we used Mean Absolute Error (MAE) calculated on the complete test dataset. To measure interpretability, we employed five key metrics from the Quantus library [5], each designed to assess different dimensions of attribution reliability and explanation quality. Due to computational constraints, we applied the interpretability evaluation to a representative sample of 10,000 instances from the test set, while performance metrics were computed across the entire test dataset to ensure robust statistical evaluation.

5.1 Performance and Interpretability Analysis

The comparison shown in Fig. 3 highlights notable differences in performance and trade-offs in interpretability across models.

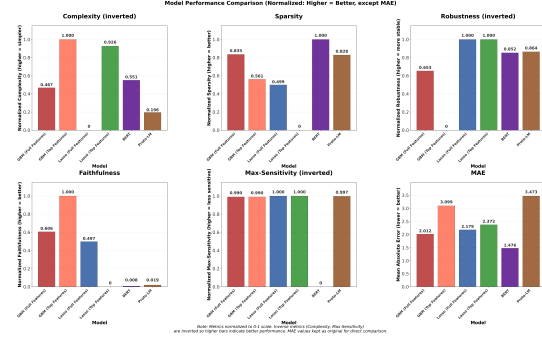


Figure 3: Models Performance Comparison. Comparison of model performance across six evaluation metrics. Bars represent normalized values (0–1) for all metrics except MAE, which retains its original scale (lower is better). For Complexity and Max-Sensitivity, lower values are better, so they were inverted to maintain consistency where higher bars always reflect better performance. The compared models include GBM and Lasso with both full and top 15 features, alongside BERT and Proto-lm.

Model Performance (MAE): The performance results reveal interesting patterns across model types. BERT emerged as the clear winner with an MAE of 1.476, demonstrating the advantage of pre-trained transformers for this task, while Proto-lm struggled significantly at 3.473 MAE. The traditional ML models showed more nuanced behavior - Lasso proved remarkably stable across feature sets, with barely any difference between using all 200 features (2.372 MAE) versus just the top 15 (2.179 MAE). This stability suggests Lasso naturally identifies and relies on the most relevant features. GBM exhibited much higher sensitivity to feature selection, decreasing dramatically from 2.012 to 3.099 MAE when restricted to top features - a substantial gain of over 1.0 point. This pattern indicates GBM may be more suitable for high-dimensional feature spaces but performs poorly when it can access a small number of features. The generally modest performance across all models suggests either our NLP task proved more challenging than anticipated or our feature extraction pipeline could benefit from refinement.

Interpretability - Complexity and Sparsity: As expected, deep learning models (BERT and Proto-lm) ranked among the most complex, alongside full-feature versions of traditional models. The most interpretable model was GBM with top 15 features, followed closely by Lasso's top 15 version. This confirms that interpretability depends on both model architecture and feature selection strategy. Sparsity patterns mirrored complexity results, suggesting that complex models with extensive feature sets don't utilize all available features during inference.

Robustness and Faithfulness: GBM achieved the highest faithfulness scores, with the top features version outperforming the full version. This aligns well with GBM's ensemble of decision

trees architecture, where deterministic tree-based decisions naturally produce consistent, faithful explanations. However, GBM performed poorly on robustness, meaning it changes predictions more frequently when inputs are slightly perturbed. This creates an interesting trade-off where the model provides reliable explanations but remains sensitive to input variations. BERT and Proto-lm performed reasonably well across both metrics, defying expectations that deep learning models would be less interpretable. Lasso also showed mixed performance in these categories, demonstrated better robustness while maintaining low faithfulness, suggesting that models can be stable against noise, yet still fail to identify truly important features for predictions.

Feature Sensitivity: Most models showed low sensitivity to noise in their most important features, with BERT as the notable exception. BERT's extremely high max-sensitivity score may reflect either a calculation anomaly or genuine hypersensitivity to token masking, warranting further investigation.

5.1.1 Instance explanations. To get a better understanding of each model's inner working we used post-hoc methods (LIME and SHAP) on the following four instances from the test set:

1. Case with rating 1: - "i've had lower back pain for over a year and several doctors later, i was put on cymbalta. i could not handle the side effects. i was having horrible dreams, sweats, felt nauseated, shakes, and felt really out of it with dizziness and confusion. this was a 30 mg dose, i'm not on any other medicines"

2. Case with rating 3: "was prescribed for kidney stone pain, and got very little relief. for me, a single recommended 10 mg pill worked about as well as 2 ibuprofen - a double dose did work a bit, but again - not better than 800 mg of ibuprofen. thankfully, i also received a prescription for hydrocodone - which, used sensibly - can take out all but the very worst kidney stone pain."

3. Case with rating 7: "i was prescribed to this medicine for morning sickness. i never vomit but always feel nauseated. it only works for me sometimes. sometimes i feel like a million bucks..other times its like i didnt even take any medicine"

4. Case with rating 10: "i have been taking beyaz for exactly a month now and my acne has cleared up so much! i have always had really bad skin and it has made it flawless! it is amazing! i love beyaz!"

5.1.2 LIME. To interpret predictions from the two deep learning models, we applied LIME on the four aforementioned test cases.

Fig. 4 shows LIME explanations for the BERT model across four test cases. For low-rating reviews (Fig. 4 A and B), BERT correctly identifies negative terms. In the rating 1 case, words like "horrible," "nausea," and "effect" are marked as decreasing the rating. Similarly, for rating 7 (C), terms such as "vomit" and "prescribe" show negative contributions. This demonstrates BERT's ability to recognize side-effect vocabulary and link it to lower ratings.

In higher rating reviews (Fig. 4 C and D), the model appropriately identifies positive sentiment. Words like "amazing," "love," and "flawless" in the rating 10 case are marked as increasing the rating, which matches expected behavior. While some neutral words receive unexpected importance scores and contribution magnitudes do not always align with intuition, BERT generally shows good

ability to distinguish between rating levels and provide reasonable explanations for its predictions.

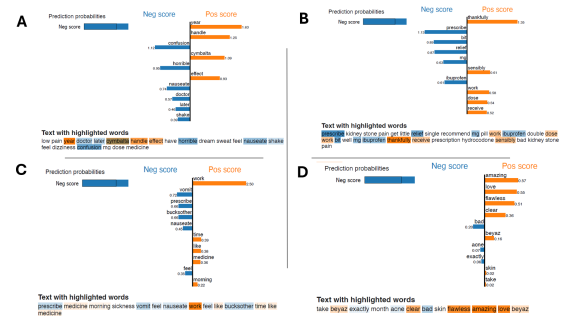


Figure 4: LIME explanations for BERT model on four test cases. (A) Rating 1 (predicted: 4), (B) Rating 3 (predicted: 7), (C) Rating 7 (predicted: 3), and (D) Rating 10 (predicted: 9). Orange bars indicate words that increase predicted ratings, blue bars decrease ratings.

Fig. 5 shows the results for the Proto-lm model. The model predicted a maximum rating of 10 in all four cases, despite true ratings being 1, 3, 7, and 10. This consistent overprediction suggests the model learned a strong bias toward the highest class, possibly due to class imbalance, even though class weights were used during training.

Looking more closely at the explanations, the feature importance scores reveal inconsistent and sometimes contradictory logic. In Fig. 5D, where the true rating was 10 and the prediction was also 10, the model correctly attributed higher importance to words like "love" and "clear" (both contributing positively to the rating). However, other highly positive words like "amazing" and "flawless" were surprisingly marked as decreasing the rating. This inconsistency undermines the model's interpretability, even in correctly classified examples.

In the misclassified cases (Fig. 5 A–C), the model more clearly associates side-effect-related terms such as "vomit", "pain", and "dizziness" with lower ratings, which aligns better with human expectations. Still, the overall pattern lacks coherence, and the identical high-confidence predictions across diverse inputs indicate a failure to capture meaningful distinctions between reviews.

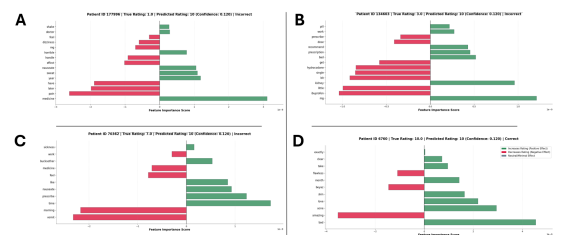


Figure 5: LIME explanations Proto-lm model for four test cases. (A) Rating 1, (B) Rating 3, (C) Rating 7, and (D) Rating 10 test cases. Green bars indicate words that increase predicted ratings, red bars decrease ratings. All cases were predicted as rating 10, demonstrating model collapse toward the majority class.

5.1.3 SHAP. In the following section, the SHAP instance explanations of the Lasso and LightGBM models will be discussed. Note that the resulting value of the SHAP explanation may differ from the predicted label, because not all features can be displayed here.

The linear regression model, as shown in Fig. 6, did not perform particularly well, especially for higher real values, as shown in the two bottom figures where predictions were relatively close but still imprecise. In contrast, predictions for lower values (shown in the top two figures) were notably inaccurate. Some SVD-derived features had stronger influence than others, with "Dosage and Symptoms" consistently ranking among the top six most important features. Interestingly, the sentiment score did not appear among the top predictors in the linear model, unlike in the GBM model, where it played a more significant role. This difference likely stems from Lasso regularization shrinking marginal feature coefficients toward zero. The SVD-derived features present interpretability challenges, as they lack clear, concrete meanings and often appear similar to each other. This makes understanding what each feature represents difficult, and even SHAP explanations remain somewhat unclear, potentially leading to confusion rather than actionable insights.

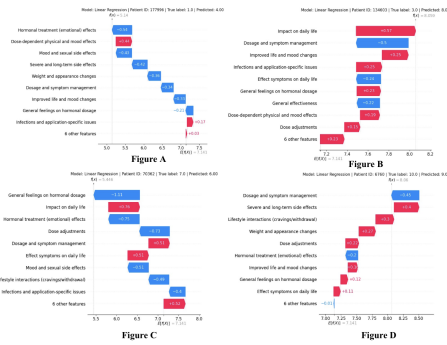


Figure 6: SHAP explanations for linear regression predictions on four test instances. Each subplot shows feature contributions with red (positive) and blue (negative) bars. (A) True label 1, predicted 4.0; (B) True label 3, predicted 8.0; (C) True label 7, predicted 6.0; (D) True label 10, predicted 9.0.

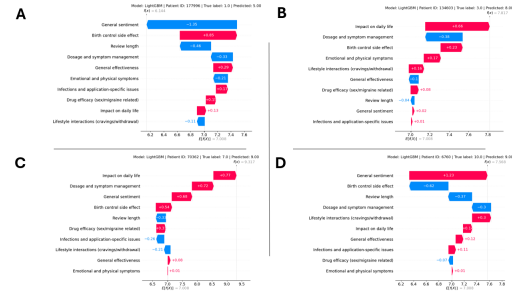


Figure 7: LightGBM SHAP explanations for the four samples. Each subplot (A–D) presents the SHAP feature contributions for a specific patient, highlighting how different features influenced the model’s prediction of a perfect rating (10), regardless of the true label. (A) Instance with true label 1 was misclassified as 10, with the model overweighing positive sentiment despite strong negative cues. (B) Instance with true label 3 was also misclassified as 10, driven by a misinterpretation of the review’s impact on daily life. (C) Instance with true label 7 shows overly positive interpretation by the model, leading to a prediction that overshoots the true rating. (D) Instance with true label 10 was correctly classified, with the model’s decision primarily shaped by the general positive sentiment in the review.

6 DISCUSSION

An interesting challenge we encountered was developing a truly interpretable model. We initially selected the LASSO model (linear regression) expecting it to provide simple and clear explanations, but discovered that achieving genuine interpretability proved far more complex than anticipated. Without careful hyperparameter tuning, the model continued to rely on hundreds of features for its predictions, rendering interpretation practically impossible.

To address this issue, we implemented SVD feature reduction, which successfully reduced dimensionality but introduced new interpretability challenges. The resulting SVD components lacked the intuitive meaning of the original TF-IDF features, and while we attempted to manually assign interpretable labels, these labels often failed to capture the true semantic content of the components. For instance, SVD feature 20, which we labeled "Effect symptoms on daily life," was composed of disparate terms including work, migraine, medicine, sex, month, year, period, effect, cramp, and mood. Finding a coherent label that accurately represented this heterogeneous collection of terms proved extremely difficult, raising questions about whether any meaningful interpretation existed at all.

The deep learning models also posed challenges. In particular, the Proto-Im approach introduced unexpected implementation hurdles that prevented us from utilizing its prototype-based explanation capabilities. Although the model holds theoretical promise for interpretability, the lack of thorough documentation and limited tutorial resources created significant obstacles. Without clear framework guidance, accessing the model’s core interpretability features proved unfeasible. This experience underscores a broader issue in the field: even methodologically sound approaches can become impractical without sufficient implementation support. It highlights

the importance of robust documentation when developing new machine learning tools.

Beyond interpretability challenges, the limited predictive performance across all four models suggests that the task itself may be inherently difficult. Predicting medication ratings solely from review text appears to be a complex undertaking that may challenge even human judgment. Reviews often contain nuanced sentiment where high ratings coexist with negative aspects mentioned in the text, and conversely, lower ratings may be accompanied by positive commentary. This inherent ambiguity in the relationship between textual content and numerical ratings may represent a fundamental limitation of text-based prediction approaches for this domain.

These experiences revealed important lessons: building effective models requires balancing multiple competing objectives-interpretability, performance, and practical implementation feasibility. Feature extraction and dimensionality reduction techniques can significantly compromise interpretability even when using theoretically transparent algorithms, while the complexity of human sentiment expression may impose natural limits on predictive accuracy.

For future work, incorporating medication type as an explicit feature could substantially improve both model performance and interpretability. While we excluded this information to ensure broader applicability-since medication details are not always available-including it could simplify the classification task considerably. Reviewers often employ medication-specific language patterns, and given that birth control represented the most common medication in our dataset, many SVD features exhibited specificity to this category. Segmenting data by medication type might enable more meaningful component extraction for other pharmaceutical categories, potentially yielding more interpretable and clinically relevant features.

7 CONCLUSION

In summary, finding a model that is both interpretable and accurate for an NLP task remains a significant challenge. Lasso was the least accurate among the models we tested and, contrary to expectations, was not particularly interpretable. The model performed poorly on metrics such as complexity and sparsity, although it scored reasonably well on others. LightGBM offered a balance between accuracy and interpretability, performing well on faithfulness, sensitivity, and sparsity, but falling short on complexity and robustness. BERT was the most accurate model overall, but - as expected - it lacked interpretability, particularly due to its high sensitivity and low faithfulness. ProtoLM had the lowest overall performance, with low accuracy and only moderate interpretability. Although it performed well in terms of sparsity and robustness, it scored poorly on complexity, faithfulness, and sensitivity.

These results highlight the challenge of developing interpretable models for NLP tasks. While we anticipated that simpler models like Lasso would be more transparent, they turned out to be less interpretable than expected. This suggests that relying solely on traditional "interpretable" methods may not be sufficient. Instead, our findings emphasize the need for new, inherently interpretable models that are both effective and user-friendly. While we support the call by Rudin [1] for prioritizing interpretability, our work

shows that the current tools often fail to deliver a satisfying balance between accuracy and interpretability in practice.

8 ETHICS

Using the provided models on medication reviews brings up some important ethical questions. First, there is a problem with bias data. Our dataset comes from only one online platform (Kaggle), so it does not represent all kinds of patients. This means our models might not work fairly for different groups of people, and the results cannot be considered for every group of people.

Another big risk is that people can misunderstand the model's output. The predicted score should not be mistaken for clinical efficacy or medical advice, as this could lead to harmful health decisions.

Finally, the main ethical consideration for our project is the trade-off between performance and transparency, an idea discussed by Cynthia Rudin. Our "black-box" BERT model was the most accurate, but because its process is opaque, it could have unsafe hidden biases. For an important area like healthcare, the safety and fairness of an interpretable model, whose logic we can check, is more important than just getting the highest possible accuracy.

9 DIVISION OF LABOR

We happily report that we collaborated closely and supported each other throughout the project. While our contributions were shared and balanced, the main tasks were divided as follows:

Table 1: Division of Labor Among Team Members

Name	Responsibility
Julia Baas	LightGBM model, features extraction and hyperparameter tuning
Bar Melinarskiy	Proto-lm model and evolution metrics
Ali Nesaei	Data Preprocessing and BERT model
Youssef Ben Mansour	Linear Regression model and features extraction

REFERENCES

- [1] Cynthia Rudin. Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. *Nature machine intelligence*, 1(5):206–215, 2019.
- [2] Sean Xie, Soroush Vosoughi, and Saeed Hassanpour. Proto-lm: A prototypical network-based framework for built-in interpretability in large language models. *arXiv preprint arXiv:2311.01732*, 2023.
- [3] Scott M. Lundberg and Su-In Lee. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, pages 4765–4774, 2017.
- [4] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. "why should i trust you?": Explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, pages 1135–1144. ACM, 2016.
- [5] Anna Hedström, Leander Weber, Daniel Krakowczyk, Dilyara Bareeva, Franz Motzkus, Wojciech Samek, Sebastian Lapuschkin, and MarinaM.-C. Höhne. Quantus: An explainable ai toolkit for responsible evaluation of neural network explanations and beyond. *Journal of Machine Learning Research*, 24(34):1–11, 2023.
- [6] Sarthak Jain and Byron C Wallace. Attention is not explanation. *arXiv preprint arXiv:1902.10186*, 2019.
- [7] Andreas Madsen, Siva Reddy, and Sarath Chandar. Post-hoc interpretability for neural nlp: A survey. *ACM Computing Surveys*, 55(8):1–42, 2022.
- [8] Mohamed Abdelwahab Ali. drug-review, 2025.
- [9] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 2623–2631. ACM, 2019.
- [10] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. Lightgbm: A highly efficient gradient boosting decision tree. *Advances in neural information processing systems*, 30, 2017.

- [11] Ibrahim Halil Sugoza, Can Verberi, and Sema Yasar. Machine learning approaches to credit risk: Evaluating turkish participation and conventional banks. *Borsa Istanbul Review*, 25(3):497–512, 2025.
- [12] William Falcon and the PyTorch Lightning team. Pytorch lightning. *GitHub*. Note: <https://github.com/Lightning-AI/lightning>, 2019.